Technical Specification

EvTron: AI-Powered Campus Event Discovery

Prepared by Sarang P

October 2026

Version 1.0

Technical Specification

EvTron: AI-Powered Campus Event Discovery Platform

Prepared by: **Sarang P**

1. Introduction

This technical specification outlines the requirements, functionality, and design for **EvTron**. The platform lets student clubs publish events by sending a poster image to a Telegram bot. An AI model reads the poster, extracts the event details, and stores them in a database. Students then browse upcoming events in a mobile-first web feed and, after signing in with Google, save their favorites. The aim is to remove the need for an admin portal and keep posting as easy as sharing a photo.

1.1 Purpose

The purpose of this document is to define the technical specification of EvTron, including the functional requirements, software and hardware requirements, data model, security, and testing approach. It serves as a blueprint for the developer and any future contributors who design, build, and maintain the platform.

1.2 Scope

This specification covers the architecture, database schema, ingestion pipeline, frontend behavior, security policies, testing, deployment, and maintenance of the EvTron MVP.

In scope: Telegram bot ingestion, AI poster extraction, event feed, Google sign-in, favorites, automated expiry of old events.

Out of scope for the MVP: a web admin portal, push or email notifications, payments or ticketing, comments or chat, native mobile apps, and integrations beyond those listed here.

1.3 Definitions and Acronyms

- **AI**: Artificial Intelligence
- **API**: Application Programming Interface
- **MVP**: Minimum Viable Product
- **OCR**: Optical Character Recognition
- **OAuth / PKCE**: Open standard for delegated sign-in, and the Proof Key for Code Exchange extension used for secure browser flows
- **RLS**: Row Level Security (PostgreSQL feature that restricts which rows a user can read or write)
- **SPA**: Single Page Application
- **UI / UX**: User Interface / User Experience
- **UTC / IST**: Coordinated Universal Time / Indian Standard Time (UTC+05:30)
- **TLS**: Transport Layer Security
- **CI/CD**: Continuous Integration and Continuous Deployment

2. System Overview

2.1 System Description

EvTron is a cloud-based application with three parts: a Python Telegram bot that ingests posters, a Supabase backend (PostgreSQL database, authentication, and file storage), and a Next.js web client. Club representatives send a poster photo to the bot. The bot sends the image to the Google Gemini API, which returns structured event data. After the sender confirms the details, the bot uploads the poster to storage and inserts the event into the database. The web client reads upcoming events from the database and shows them in a scrolling feed.

2.2 Assumptions

- The platform runs on managed cloud services (Supabase, Vercel, and a worker host such as Railway, Render, or Fly.io).
- Students use the product mainly on mobile browsers, so the UI is designed mobile-first (375 px width as the baseline).
- Browsing events requires no account; signing in is needed only to save favorites.
- Only approved club accounts can post events through the bot.
- All event times are in India (IST) and are stored in UTC.
- The free tiers of Supabase, Vercel, and Gemini are enough for the expected MVP traffic.

2.3 Dependencies

- **Cloud and hosting:** Supabase (database, auth, storage), Vercel (frontend hosting), Railway, Render, or Fly.io (24/7 bot worker)
- **Third-party services:** Telegram Bot API, Google Gemini API, Google OAuth
- **Frameworks and libraries:** Next.js, React, Tailwind CSS, `@supabase/supabase-js`, `python-telegram-bot` (v20+), `google-genai`, `supabase` (Python client), `python-dotenv`, `pillow`
- **Design:** UI designed manually in Figma and implemented by hand

3. Functional Requirements

3.1 Event Browsing

- **FR-001**: The system must show a public feed of upcoming events without requiring sign-in.
- **FR-002**: The feed must include only events whose `event_end` is in the future, sorted by `event_start` ascending.
- **FR-003**: Each event card must show the poster, title, club name, date and time in IST, location, and RSVP link when available.
- **FR-004**: The feed must show clear loading, empty, and error states.

3.2 Authentication and Favorites

- **FR-005**: The system must support sign-in with Google through Supabase Auth using the OAuth PKCE flow.
- **FR-006**: A signed-in user must be able to save and unsave an event with a heart button.

- **FR-007**: The heart button must use optimistic UI: the state toggles instantly, the database write runs in the background, and the state rolls back with an error toast if the write fails.
- **FR-008**: If a signed-out user taps the heart, the system must store the `event_id` in local storage, start Google sign-in, and save the event automatically after sign-in completes.
- **FR-009**: A signed-in user must be able to view a "Saved" page listing only their favorite events.

3.3 Telegram Ingestion

- **FR-010**: The bot must accept poster images sent as photos or as image files (documents), since photo compression can hurt text recognition.
- **FR-011**: The bot must only act on messages from Telegram users listed in the `clubs` allowlist and ignore everyone else.
- **FR-012**: The bot must send the image to the Gemini API and receive structured JSON with `title`, `club_name`, `event_start`, `event_end`, `location`, and `rsvp_link`. Missing values must be `null`.
- **FR-013**: If `event_end` is missing, the system must default it to `event_start` plus 3 hours.
- **FR-014**: The bot must validate that `event_start` is in the future and within about one year, and ask the user to correct it if not.
- **FR-015**: The bot must reply with the parsed details and Confirm / Cancel buttons, and insert the event only after Confirm.
- **FR-016**: The bot must upload the poster to the `posters` storage bucket and store its public URL on the event.
- **FR-017**: The bot must prevent duplicate events using the Telegram file identifier.
- **FR-018**: A club must be able to remove its own most recent event with a `/delete` command.

3.4 Lifecycle and Maintenance

- **FR-019**: Past events must be hidden from the feed automatically through the query filter.
- **FR-020**: A weekly scheduled job must permanently delete events that ended more than 30 days ago.

4. System Architecture

4.1 High-Level Design

The system uses a simple three-part architecture with Supabase as the shared backend. The bot and the web client never talk to each other directly. They communicate only through the database and storage.

```
Club rep --> Telegram --> Python bot --> Gemini API (poster to JSON)
                                |
                                v
                    Supabase (PostgreSQL + Storage + Auth)
                                ^
                                |
Student --> Next.js web app (Vercel)
```

4.2 Modules

- **Frontend Client (Next.js / React):** A mobile-optimized SPA with the event feed, event cards, authentication flow, and saved-events page. It uses the public `anon` key only.
- **Ingestion Bot (Python):** A long-running polling process that receives posters, calls Gemini, runs the confirmation flow, and writes to Supabase using the service role key.
- **Backend (Supabase):** PostgreSQL storage, RLS policies, Google authentication, and the public `posters` bucket.
- **Scheduler (`pg_cron`):** Weekly cleanup of old events.

4.3 Database

The system uses PostgreSQL (through Supabase) with the following tables.

clubs (allowlist of approved posters)

Column	Type	Notes
`telegram_user_id`	BIGINT	Primary key
`club_name`	TEXT	Not null; used as the reliable club name

events

Column	Type	Notes
`id`	UUID	Primary key, auto-generated
`title`	TEXT	Not null; extracted by AI
`club_name`	TEXT	From the `clubs` table
`event_start`	TIMESTAMPTZ	Not null; stored in UTC
`event_end`	TIMESTAMPTZ	Not null; used for expiry
`location`	TEXT	Venue or virtual link
`poster_url`	TEXT	Public storage URL
`rsvp_link`	TEXT	External registration URL
`telegram_file_id`	TEXT	Unique; prevents duplicates
`created_at`	TIMESTAMPTZ	Default `now()`

Indexes: `event_start` and `event_end`.

users

Column	Type	Notes
`id`	UUID	Primary key; mirrors the Supabase Auth user ID
`email`	VARCHAR	From Google OAuth
`created_at`	TIMESTAMPTZ	Default `now()`

A database trigger fills this table automatically when a new `auth.users` row is created.

saved_events (junction table)

Column	Type	Notes
`id`	UUID	Primary key
`user_id`	UUID	Foreign key to the user; cascade delete
`event_id`	UUID	Foreign key to `events.id`; cascade delete

Constraint: unique `(user_id, event_id)` to prevent duplicate bookmarks.

Feed query:

```
SELECT * FROM events WHERE event_end > now() ORDER BY event_start ASC;
```

5. Hardware Requirements

The platform runs entirely on managed cloud services, so no dedicated servers are needed.

5.1 Minimum Hardware Specifications

Cloud (bot worker)

- **CPU:** 1 shared vCPU
- **Memory:** 512 MB RAM
- **Storage:** 1 GB
- **Network:** Standard outbound internet access (the bot uses polling, so no public inbound URL is required)

Developer machine

- **CPU:** Any modern dual-core 64-bit processor
- **Memory:** 8 GB RAM
- **Storage:** 10 GB free disk space
- **Network:** Stable broadband connection

5.2 Recommended Hardware

Cloud (bot worker)

- **CPU:** 1 dedicated vCPU
- **Memory:** 1 GB RAM
- **Storage:** 5 GB
- **Network:** Always-on instance with automatic restart on crash

Developer machine

- **CPU:** Quad-core 64-bit processor
- **Memory:** 16 GB RAM
- **Storage:** SSD with 20 GB or more free space
- **Network:** Stable broadband connection

6. Software Requirements

6.1 Operating System

The bot runs on a Linux-based container image (for example Ubuntu or Debian) on the chosen worker host. Development can be done on Windows, macOS, or Linux. The frontend is served by Vercel and does not depend on a specific server operating system.

6.2 Software Dependencies

- **Programming languages:** Python 3.11 or newer (bot), TypeScript with Node.js 20 or newer (frontend)
- **Frontend:** Next.js (App Router), React, Tailwind CSS, `@supabase/supabase-js`
- **Bot:** `python-telegram-bot` v20+, `google-genai`, `supabase`, `python-dotenv`, `pillow`
- **Database and backend:** PostgreSQL via Supabase, with the `pg_cron` extension
- **Tooling:** Git, GitHub, VS Code, Figma (design)

6.3 Security Protocols

- All communication must use HTTPS (TLS 1.2 or higher), as provided by Supabase, Vercel, and Telegram.
- Data is encrypted at rest by the managed providers.
- Secrets (bot token, Gemini key, Supabase service role key) live only in `.env` files or host environment variables. They are never committed to Git and never placed in the frontend.
- The frontend uses only the public `anon` key and relies on RLS for protection.
- Row Level Security is enabled on every table:
  - `events`: public `SELECT`; writes only through the service role key used by the bot.
  - `saved_events`: `SELECT`, `INSERT`, and `DELETE` allowed only where `auth.uid() = user_id`.
  - `clubs`: no public policies; readable only by the bot.
- Sign-in uses Google OAuth with PKCE, so the app stores no passwords.
- The bot ignores any Telegram user not in the `clubs` allowlist.

7. Data Management

7.1 Data Storage

Structured data (events, clubs, users, saved events) is stored in Supabase PostgreSQL. Poster images are stored in a public Supabase Storage bucket named `posters`. All timestamps are stored in UTC and displayed in IST on the client using `Intl.DateTimeFormat`.

7.2 Data Backup

Supabase provides managed database backups according to the project's plan. The SQL schema and setup scripts are kept in the Git repository so the database can be recreated if needed. Event data is short-lived and can be re-posted, so the MVP does not need a custom backup system.

7.3 Data Privacy

The only personal data collected is the email address from Google sign-in, plus the list of events each user saves. Event posters are public by design. Users can access only their own saved events through RLS. Deleting a user removes their saved events through cascading deletes. The team should review applicable privacy requirements (for example India's Digital Personal Data Protection Act) and publish a short privacy notice before launch.

7.4 Data Retention

Expired events are hidden from the feed immediately and permanently deleted by a weekly `pg_cron` job once they ended more than 30 days ago:

```
DELETE FROM events WHERE event_end < now() - interval '30 days';
```

Poster files for deleted events remain in storage unless removed separately. This is acceptable for the MVP and can be automated later.

8. Testing and Validation

8.1 Unit Testing

Bot helper functions, such as date parsing, the 3-hour default end time, and future-date validation, are tested with `pytest`. Frontend date formatting and card components can be tested with Jest or Vitest.

8.2 Performance Testing

Targets for the MVP:

Area	Target
Feed first load on mobile (4G)	Under 3 seconds
Poster to Confirm prompt (including AI extraction)	Under 15 seconds
Favorite toggle (visual feedback)	Instant (optimistic UI)
Expected load	Hundreds of concurrent students

8.3 AI Extraction Validation

Before launch, test the Gemini prompt with 10 to 15 real posters of varying quality, including cluttered and low-resolution designs. Refine the prompt until most posters are extracted correctly. Track the common failures (wrong year, missing end time, misread venue), since the confirmation step is the safety net for these.

8.4 User Acceptance Testing (UAT)

A small group of club representatives and students will test the full flow on real phones: send a poster, confirm, see it in the feed, sign in, and favorite it. Feedback is collected before a wider campus launch.

8.5 Security Testing

Verify that:

- An anonymous user cannot insert, update, or delete events.
- A user cannot read or modify another user's saved events.
- A Telegram user outside the allowlist gets no response and creates no data.
- The service role key does not appear anywhere in the frontend code or build output.

9. Deployment and Maintenance

9.1 Deployment Plan

1. **Database:** Run the schema SQL, RLS policies, trigger, and `pg_cron` job in the Supabase project.
2. **Frontend:** Push to GitHub and import into Vercel with the public environment variables. Add the production URL to Supabase's auth redirect list and to the Google OAuth client.
3. **Bot:** Deploy to Railway, Render, or Fly.io as a worker process (start command `python bot.py`) with its environment variables. Only one polling instance can run at a time, so stop any local copy first.
4. **Release:** Start with a few clubs as early adopters, then open to the wider campus. Vercel redeploys automatically on every push to the main branch.

9.2 Maintenance

- Review bot logs weekly for extraction failures and errors.
- Keep dependencies and the Gemini SDK up to date.
- Add or remove clubs in the `clubs` table as needed.
- Rotate API keys if they are ever exposed.

9.3 Monitoring

- Bot: platform logs and restart-on-failure from the hosting provider, plus Python `logging`.
- Frontend: Vercel analytics and error logs.
- Database: Supabase dashboard for usage, storage, and query logs.
- Optional: a free uptime monitor to alert if the bot or site stops responding.

9.4 Implementation Timeline

Phase	Focus	Estimate
0. Setup	Tools and accounts	About half a day
1. Database	Supabase schema, RLS, storage	About half a day
2. Telegram bot	Echo bot, photo download, Gemini, Supabase save	3 to 5 days
3. Frontend feed	Figma to Next.js, fetch and display events	1 to 2 weeks
4. Auth and favorites	Google login, heart button, saved page	About 1 week
5. Deploy	Vercel, bot hosting, `pg_cron` cleanup	2 to 3 days

Total: about 4 to 6 weeks part-time.

9.5 Risks and Mitigations

Risk	Mitigation
AI misreads a poster	Confirm / Cancel step, `/delete` command, date validation
Strangers post to the bot	`clubs` allowlist
Timezone bugs	Store UTC, display IST, never format dates by hand
Leaked API keys	`.gitignore`, environment variables, service role key kept out of the frontend
Bot goes offline	Always-on worker with auto-restart and log monitoring
OAuth redirect errors	Register both localhost and production URLs in Google and Supabase

10. Conclusion

EvTron turns a simple action, sending a poster to a Telegram bot, into a searchable and shareable event feed for students. The architecture keeps the MVP small and low-cost: Supabase handles data, auth, and storage, a lightweight Python bot handles ingestion, and a mobile-first Next.js app handles discovery. With the safeguards described here (club allowlist, confirmation step, RLS, and automatic expiry), the platform can launch safely on campus and grow with new features such as reminders and a web posting portal.